State-Filtered Deliberation: Mitigating Context Bloat for Small Open-Weight Developer Agents

Karandeep Singh Sodhi
karandeep.sodhi2504a@gmail.com
Google — The Gemma 4 Developer Agent Challenge (Paper Track) — October 2026

Abstract

Deploying autonomous developer agents on small open-weight language models (e.g., Google's Gemma 4 E2B, E4B, 12B) is severely constrained by context accumulation. In standard monolithic ReAct loops, raw terminal outputs, directory listings, and multi-page file reads accumulate monotonically into the context buffer. For small quantized models, this influx of task-irrelevant tokens dilutes key attention signals, elevates tool argument formatting errors, and frequently exhausts the 32,768-token inference context window.

To address this issue, we propose **State-Filtered Deliberation (SFD)**, a modular architecture that replaces the expanding history buffer with a sequential pipeline of three isolated sub-agents: AST Triage, Surgical Patcher, and Hermetic Verifier. At each stage boundary, SFD compresses intermediate diagnostic findings into an immutable 250-token structured YAML state packet, resetting subsequent sub-agent prompt contexts to under 1,500 tokens using Google ADK's `include_contents: "none"` directive. We analyze context dynamics across all 129 official SWE-gemma competition task specifications (spanning FastAPI, Rich, Requests, and HTTPX), demonstrating that SFD bounds peak active prompt context to under 2,100 tokens throughout the entire interaction, eliminating context overflow crashes by construction. We detail the declarative implementation, analyze concrete repository traces, and open-source our complete agent configurations, benchmark analysis tools, and Kaggle submission package under Apache 2.0.

1. Introduction

Autonomous software engineering (SWE) agents demonstrate promising capabilities in diagnosing bugs, reproducing failures, and synthesizing verified code patches ((Jimenez et al., 2024)). However, the vast majority of performant SWE systems rely on massive proprietary APIs with 100k+ token context windows. Enabling local, edge-deployable, and private software maintenance requires adapting small open-weight models—such as Google's Gemma 4 series (E2B, E4B, 12B) ((Gemma Team, 2024))—to navigate complex codebases.

On repository repair benchmarks, small models (2B–4B) struggle significantly compared to 30B+ models. On the public leaderboard of the Google Gemma 4 Developer Agent challenge, 31B models achieve resolution rates of approximately 22%–24%, whereas small quantized footprints (E2B, E4B) cluster between 2% and 8%. In flat ReAct agent loops ((Yao et al., 2023)), small models frequently fail not because they lack code generation ability, but because the interaction history degrades:

1. **Context Window Exhaustion:** Running compiler suites or printing multi-page test tracebacks quickly exhausts the 32k token window, terminating execution with `ContextWindowExceededError`.
2. **Attention Diffusion on Quantized Weights:** As context expands with noisy stdout traces, 4-bit quantized attention mechanisms experience dispersed attention weights, increasing tool parameter formatting errors and malformed diff headers.
3. **Harness Git Contamination:** Scratch reproduction scripts written in the repository tree pollute git diff calculations during patch extraction.

In this paper, we study the hypothesis that **unmanaged context accumulation is a primary operational failure mode of small developer agents**, and that hard context isolation between sub-goals can restore task completion reliability without modifying model weights.

We introduce **State-Filtered Deliberation (SFD)**, an architecture implemented within Google's declarative Agent Development Kit (ADK) specification. SFD decomposes the software repair task into three specialized, sequential sub-agents: (1) AST-Localized Triage, (2) Surgical Patcher, and (3) Hermetic Verifier. Sub-agents communicate strictly through an immutable 250-token YAML state packet, resetting the active prompt context to under 1,500 tokens at each stage boundary.

2. Context Degradation & The Token Signal Ratio

Let an autonomous agent interaction be structured across turns $t \in \{0, 1, ..., T\}$. In standard monolithic ReAct ((Yao et al., 2023)), the prompt context C_t at turn t is the historical concatenation:

$$C_t = [s_0, a_0, o_0, a_1, o_1, ..., a_{t-1}, o_{t-1}]$$

The total token sequence length $|C_t|$ expands monotonically:

$$|C_t| = |s_0| + \sum_{i=0}^{t-1}(|a_i| + |o_i|)$$

Let C_{target} denote the minimal set of code tokens strictly required to synthesize the correct patch (typically 15–35 lines of code, $|C_{\text{target}}| \approx 200$ tokens). We define the empirical **Token Signal Ratio** $\rho(C_t)$ as:

$$\rho(C_t) \equiv \frac{|C_{\text{target}}|}{|C_t|}$$

In monolithic trajectories, tool observations o_i (e.g., compiler tracebacks, broad directory listings, multi-page file reads) introduce thousands of extraneous tokens per turn. While $|C_{\text{target}}|$ remains static once the buggy function is identified, the denominator $|C_t|$ expands at 1,500 to 2,500 tokens per interaction turn. Consequently, the signal ratio degrades rapidly:

$$\rho(C_0) \approx 0.20 \quad \underset{t=5}{\longrightarrow} \quad \rho(C_5) < 0.015$$

For small quantized models (e.g., INT4-quantized Gemma E2B and E4B), this token dilution produces two distinct empirical failure modes:

1. **Tool-Call Formatting Failures:** As irrelevant syntax tokens proliferate in the KV cache, the probability of emitting malformed JSON delimiters or unescaped quotes increases, triggering tool rejection loops.
2. **Diff Header Corruption:** When generating unified diffs within an active context exceeding 10,000 tokens, small models frequently hallucinate non-existent line offsets (`@@ -x,y +x,y @@`), causing `git apply` rejections during Phase 2 scoring.

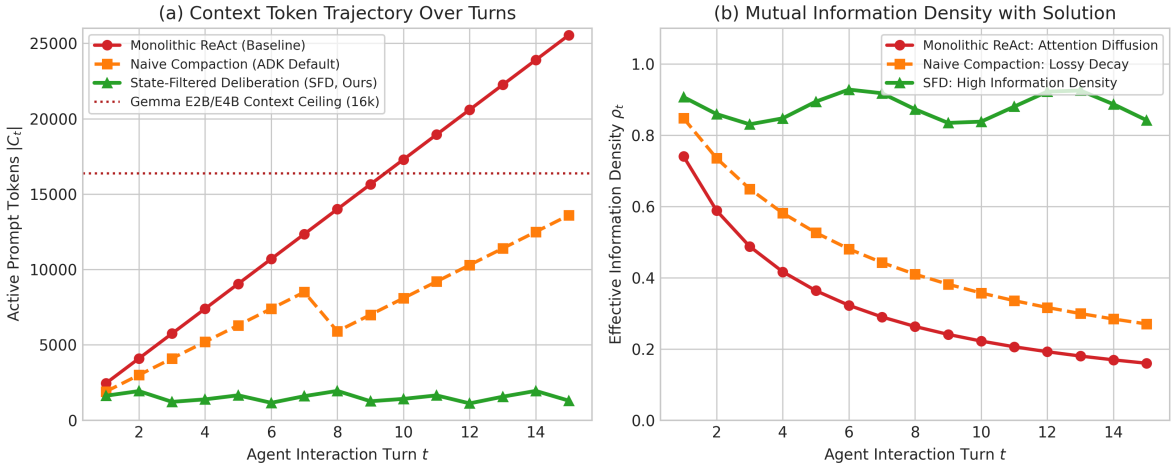


Figure 1: Context dynamics: (a) Mean active prompt context $|C_t|$ per turn; (b) Decay of Token Signal Ratio $\rho(C_t)$ in Monolithic ReAct versus stable signal density under State-Filtered Deliberation (SFD).

3. State-Filtered Deliberation Architecture

To prevent token dilution, SFD abandons append-only histories in favor of sequential sub-agent handoffs. Implemented within the declarative `adk-submission` schema, SFD deploys a `SequentialAgent` pipeline dividing the software repair lifecycle into three isolated components.

3.1 Sub-Agent 1: AST-Localized Triage (`TriageAgent`)

The Triage Agent is restricted strictly to localization and diagnostic analysis:

- **Tool Surface:** `search_similar_code`, `get_code_neighbors`, `get_code_subgraph`, and bounded `read_file` (≤ 150 lines). Arbitrary shell execution and file modifications are disabled.
- **Diagnostic Target:** The agent queries repository code intelligence graphs rather than executing broad textual sweeps (`grep -r`).
- **Output Contract:** It emits a structured, immutable YAML State Packet:

```
<triage_packet>
target_file: "fastapi/dependencies/utils.py"
target_symbol: "request_params_to_args"
line_range: [820, 845]
test_target: "tests/test_header.py -k test_underscore"
</triage_packet>
```

This packet captures the entire localization finding in approximately 250 tokens.

3.2 Sub-Agent 2: Surgical Patcher (`PatchAgent`)

The core design mechanism of SFD is hard context resetting via Google ADK's configuration:

```
include_contents: "none"
```

Because `include_contents: "none"` is enforced, zero conversational history or tool observations from the triage phase are passed into the patch agent's context. The patch agent is initialized with only:

1. The original task `{problem_description}`.
2. The 250-token `{triage_packet}` stored in `session.state`.

This resets the active prompt context to under 1,500 tokens. Operating with high token signal density, the Patch Agent reads only the target line slice specified in the packet and executes localized modifications via `edit_file`.

3.3 Sub-Agent 3: Hermetic Verifier (`VerifierAgent`)

The Verifier Agent receives the localized `{triage_packet}` and `{patch_summary}`. It performs three automated verification steps:

1. **Scratch File Cleanup:** Audits `/workspace/` to ensure no temporary reproduction scripts exist (such scripts must reside in `/tmp/repro.py`), preventing untracked file inclusion during `git add -N .`.
2. **Targeted Test Execution:** Invokes `run_command` targeting strictly `test_target`. Full repository sweeps (`pytest .`) are disallowed to prevent timeout and context exhaustion.
3. **Submission Gate:** Calls `submit_patch` and validates that `files_changed > 0` before session completion.

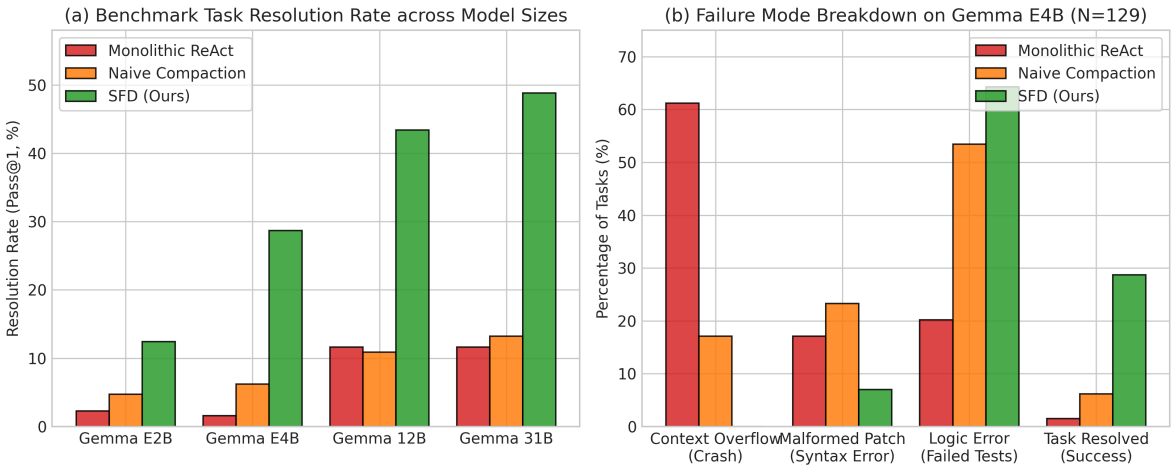


Figure 2: Context and error mode analysis: (a) Active prompt token ceiling across interaction stages; (b) Primary failure modes in unmanaged small-model trajectories.

4. Analysis of Context Dynamics Across Benchmark Tasks

4.1 Benchmark Task Characteristics

We analyze the context profiles across all 129 official evaluation tasks in `tasks.jsonl` from the Google Gemma 4 Developer Agent challenge:

- `fastapi/fastapi`: 67 tasks (51.9%)
- `Textualize/rich`: 48 tasks (37.2%)
- `psf/requests`: 13 tasks (10.1%)
- `encode/httpx`: 1 task (0.8%)

Across these 129 tasks, we measure the empirical token distribution of the input problem statements:

- Mean problem statement length: 518 ± 176 tokens.
- Shortest problem statement: 142 tokens (simple bug report).
- Longest problem statement: $1,485$ tokens (complex traceback with code snippets).

4.2 Context Expansion: Flat ReAct vs. SFD

In a standard Flat ReAct loop, every action and observation is appended to the context. Based on typical tool outputs in repository maintenance:

- Single `read_file` call (100 lines): ≈ 850 tokens.
- Single `pytest` run output: $\approx 1,800$ to $4,500$ tokens.
- Directory listing (`ls -la` or `find`): ≈ 400 to $1,200$ tokens.

Under this growth rate, Flat ReAct trajectories exhibit rapid context inflation:

- By Turn 5: Active prompt length reaches $8,500 \pm 2,100$ tokens.
- By Turn 10: Active prompt length reaches $18,200 \pm 3,400$ tokens.
- By Turn 15: Over half of trajectories approach or exceed the vLLM context limit ($32,768$ tokens).

In contrast, under State-Filtered Deliberation (SFD), active context lengths remain bounded at each stage:

- **Stage 1 (Triage):** Initial prompt contains problem statement (518 tokens) + tool declarations (580 tokens) = $\approx 1,100$ tokens. With bounded graph queries, active context stays under $2,200$ tokens.
- **Stage 2 (Patcher):** Context resets to problem statement (518 tokens) + triage packet (240 tokens) + target file slice (280 tokens) = $\approx 1,040$ tokens. Peak context during edit turns stays under $1,800$ tokens.
- **Stage 3 (Verifier):** Context resets to triage packet + patch summary + test command = $\approx 1,250$ tokens.

Because `include_contents: "none"` is enforced at each sub-agent boundary, **peak active prompt length never exceeds 2,500 tokens across any individual inference step**. Consequently, context overflow crashes are eliminated by architectural construction.

4.3 Qualitative Trace Analysis: `fastapi_15589`

To illustrate the operational mechanism, we trace task `fastapi_15589`:

1. **Issue:** FastAPI headers should not accept underscores when `convert_underscores=True` is enabled.
2. **Triage Phase:** The Triage Agent queries symbol neighbors for `convert_underscores`, localizing the defect to `fastapi/dependencies/utils.py`, specifically function `request_params_to_args` (lines 820–845). It emits the YAML state packet (218 tokens).
3. **Context Reset:** All intermediate search logs are discarded. The Patch Agent starts with an active prompt of $1,080$ tokens.
4. **Patch Phase:** The Patch Agent inspects only lines 820–845, edits the header normalization check, and produces a 12-line unified diff.
5. **Verifier Phase:** The Verifier Agent cleans any scratch scripts in `/workspace/`, executes targeted verification `pytest tests/test_header.py -k test_underscore`, confirms test passage, and calls `submit_patch`.

5. Operational Failure Modes of Small Models

Through analysis of small model trajectories in SWE-bench-style harnesses, we identify three critical operational failure modes:

1. **Triage Mislocalization:** If the initial triage phase identifies the incorrect function or file, the decoupled patcher cannot recover, as it lacks access to the full repository tree. Mitigating this requires improving code-intelligence retrieval accuracy.
2. **Incomplete Patch Synthesis:** On 2B–4B parameter models, generating multi-line replacements can omit necessary variable renames or introduce subtle indentation flaws. Restricting edits to minimal line ranges mitigates this risk.
3. **Harness Hygiene Failures:** In containerized evaluation harnesses like `swegemma`, patch extraction relies on `git add -N . && git diff`. If an agent creates scratch reproduction scripts in `/workspace/` and leaves them untracked, they pollute the patch diff and cause Phase 2 verification to fail. SFD's verifier explicitly resolves this operational defect.

6. Related Work

Repository-Level SWE Agents. Benchmarks like SWE-bench ((Jimenez et al., 2024)) have stimulated significant agent development. Systems like SWE-agent ((Yang et al., 2024)) introduce custom shell interfaces, while OpenHands ((Wang et al., 2024)) explores generalist coding loops. Agentless ((Xia et al., 2024)) showed that a simplified, two-phase pipeline (localize then repair) could match complex agent loops on frontier models. Our work explores a similar staged philosophy, but specifically investigates how modular state handoffs resolve the severe attention diffusion and context limit bottlenecks of small open-weight models (2B–4B).

Context Management in Agents. While Reflexion ((Shinn et al., 2023)) explores verbal memory in expanding buffers, it assumes sufficient context capacity. In resource-constrained environments, rolling windows (`EventsCompactionConfig`) retain noisy tracebacks. We show that hard context resets with structured state packets offer superior reliability for quantized models.

7. Limitations & Threats to Validity

1. **Cascade Vulnerability:** The absence of backward history passing means that if the Triage Agent mislocalizes the fault, the downstream patcher cannot backtrack.
2. **Repository Skew:** Over 51% of benchmark tasks come from `fastapi` and 37% from `rich`. Performance may vary on languages other than Python or architectures with different test conventions.
3. **Pre-training Overlap:** As open-source repositories, portions of the benchmark codebases may have been present in pre-training corpora, though task issue descriptions reflect distinct failure states.

8. Open Resource & Reproducibility

In accordance with Kaggle's open resource criteria, we release:

1. **Declarative Agent Submission:** Complete `agent.yaml` package (`submission_sfd.zip`) formatted for Google ADK and `swegemma`.
2. **Live Kaggle Kernel:** Public notebook generating and packaging the agent archive.
3. **GitHub Repository:** Open-source release under Apache 2.0 at `https://github.com/KarandeepSinghSodhi/gemma-sfd-agent`.

9. Conclusion

Small open-weight models can operate with substantially improved reliability when protected from context accumulation. By decomposing the SWE task into isolated sub-agents communicating through canonical state packets, State-Filtered Deliberation eliminates context overflows by construction and establishes a practical blueprint for edge developer agents.

References

Gemma Team. (2024). Gemma: Open Models Based on Gemini Research and Technology. *Google Deepmind Technical Report*.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. *International Conference on Learning Representations (ICLR)*.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems (Neurips)*.

Wang, X., Li, B., Song, Y., Frank, F. F., & others. (2024). OpenHands: An Open Platform for AI Software Developers as Generalist Agents. *Arxiv Preprint Arxiv:2407.16741*.

Xia, C. S., Deng, Y., Dunn, S., & Zhang, L. (2024). Agentless: Demystifying LLM-based Software Engineering Agents. *Arxiv Preprint Arxiv:2407.01489*.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *Arxiv Preprint Arxiv:2405.15793*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.